

Complexidade computacional

Aula 14

Fábio Henrique Viduani Martinez

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Projeto e Análise de Algoritmos I

Conteúdo da aula

- 1 Motivação
- 2 Problemas computacionais
- 3 Algoritmos polinomiais
- 4 Problemas polinomiais e a classe P
- 5 A classe NP de problemas
- 6 A questão “ $P = NP$?”
- 7 Reduções e a complexidade relativa de problemas
- 8 Problemas NP-completos

Introdução

- ▶ complexidade de algoritmos *versus* complexidade de problemas
- ▶ a **complexidade de um problema computacional** é o consumo de tempo do melhor algoritmo possível para o problema
- ▶ o melhor algoritmo conhecido para um problema muitas vezes não é o melhor algoritmo possível
- ▶ consumo de tempo no pior caso

Introdução

- ▶ quase todos os algoritmos que estudamos até o momento são **algoritmos de tempo polinomial**: sobre entradas de tamanho n , o tempo de execução de pior caso desses algoritmos é $O(n^k)$, para alguma constante k
- ▶ será que *todos* os problemas podem ser solucionados em tempo polinomial?
- ▶ problemas que são solucionados em tempo polinomial são chamados de **tratáveis**, enquanto que problemas solucionados em tempo superpolinomial são chamados de **intratáveis**

Problemas de busca/decisão

Quadrado perfeito: dado um número natural n , encontrar um número natural x tal que $x^2 = n$ (ou constatar que tal x não existe)

Equação inteira do segundo grau: dados números inteiros a, b e c , encontrar um número inteiro x tal que $ax^2 + bx + c = 0$ (ou constatar que tal x não existe)

Fatoração: dado um número natural n , encontrar um número natural p , maior que 1 mas menor que n , que seja divisor de n (ou constatar que tal p não existe)

Problemas de busca/decisão

Divisor comum grande: dados números naturais m, n e k , encontrar um divisor comum de m e n que seja maior que k (ou constatar que tal divisor não existe)

Subsequência crescente longa: dada uma sequência $s_1, \dots, s_n$ de números naturais e um número natural k , encontrar uma subsequência crescente de $s_1, \dots, s_n$ que tenha comprimento maior que k (ou constatar que tal subsequência não existe)

Soma de subconjunto: dados números naturais $p_1, \dots, p_n$ e c , encontrar um subconjunto K de $\{1, \dots, n\}$ tal que a soma dos p_k para k em K seja igual a c (ou constatar que um tal K não existe)

Problemas de busca/decisão

Mochila valiosa: dados números naturais $w_1, \dots, w_n$, $v_1, \dots, v_n$, W e d , encontrar um subconjunto K de $\{1, \dots, n\}$ tal que a soma dos w_k para k em K não passe de W e a soma dos v_k para k em K seja maior que d (ou constatar que um tal conjunto não existe)

Caminho curto: dados vértices r e s de um grafo e um número d , encontrar um caminho de r a s no grafo que tenha comprimento menor que d (ou constatar que não existe tal caminho)

Caminho longo: dados vértices r e s de um grafo e um número d , encontrar um caminho de r a s que tenha comprimento maior que d (ou constatar que tal caminho não existe)

Problemas de busca/decisão

Circuito longo: dado um grafo e um número k , encontrar um circuito de comprimento maior que k (ou constatar que o grafo não tem tal circuito)

Circuito hamiltoniano: encontrar um circuito hamiltoniano, ou seja, um circuito que passe por todos os vértices, num grafo dado (ou constatar que tal circuito não existe)

Problemas de busca/decisão

Clique grande: dado um grafo e um número k , encontrar uma clique com mais que k vértices (ou constatar que tal clique não existe)

Cobertura pequena: dado um grafo e um número k , encontrar uma cobertura com menos que k vértices (ou constatar que tal cobertura não existe)

Campo minado: dada uma configuração do jogo campo minado (*minesweeper*), encontrar uma disposição de minas que seja consistente com a configuração (ou constatar que uma tal disposição não existe)

Problemas de otimização

Máximo divisor comum: encontrar o maior divisor comum de dois números inteiros positivos m e n

Subsequência crescente máxima: dada uma sequência $s_1, \dots, s_n$ de números naturais, encontrar uma subsequência crescente máxima de $s_1, \dots, s_n$

Mochila ótima: dados números naturais $w_1, \dots, w_n$, $v_1, \dots, v_n$, e W , encontrar um subconjunto K de $\{1, \dots, n\}$ tal que a soma dos w_k para k em K não passe de W e a soma dos v_k para k em K seja máxima

Problemas de otimização

Caminho mínimo: dados vértices r e s de um grafo, encontrar um caminho de comprimento mínimo de r a s no grafo (ou constatar que não há caminho algum de r a s)

Caminho máximo: dados vértices r e s de um grafo, encontrar um caminho de r a s que tenha comprimento máximo (ou constatar que não há caminho algum de r a s)

Circuito máximo: encontrar um circuito de comprimento máximo num grafo dado (ou constatar que o grafo não tem circuito algum)

Problemas de busca *versus* problemas de otimização

- ▶ há uma relação íntima entre os problemas de otimização e os problemas de busca/decisão
- ▶ muitos problemas de otimização podem ser “reduzidos” aos correspondentes problemas de decisão/busca

Algoritmos “rápidos”

- ▶ um algoritmo **resolve** um dado problema computacional se, ao receber qualquer instância do problema, devolve uma solução da instância ou informa que a instância não tem solução
- ▶ um algoritmo que resolve um dado problema é **polinomial** se o seu *consumo de tempo no pior caso* é limitado por um polinômio no *tamanho* das instâncias do problema
- ▶ ou seja, o algoritmo é polinomial se existe um número k tal que o consumo de tempo do algoritmo é $O(n^k)$, sendo n o tamanho da instância

Algoritmos “rápidos”

- ▶ por exemplo, é polinomial todo algoritmo que consome no máximo $100n^4 + 300n^2 + 5000$ unidades de tempo, sendo n o tamanho da instância
- ▶ no caso do problema da fatoração, o tamanho de uma instância é o número de bits/dígitos de n , ou seja, cerca de $\lg n$
- ▶ algoritmos polinomiais são considerados rápidos (ainda que seja difícil aceitar como rápido um algoritmo que consome tempo proporcional a n^{500} , por exemplo) e algoritmos não polinomiais são considerados inaceitavelmente lentos (como por exemplo, os que consomem tempo 2^n ou 1.1^n , ainda que possam ser úteis para valores muito modestos de n)

Problemas “fáceis”

- ▶ um problema computacional é **polinomial** se existe um algoritmo polinomial para o problema
- ▶ problemas desse tipo são considerados **tratáveis** ou **fáceis**
- ▶ o problema da equação do segundo grau, o problema do divisor comum grande, o problema da subsequência crescente longa, o problema do caminho curto são exemplos de problemas polinomiais
- ▶ a **classe P** de problemas é o conjunto de todos os problemas polinomiais

Problemas não polinomiais

- ▶ um problema é **não polinomial** se não existe algoritmo polinomial que resolva o problema
- ▶ problemas desse tipo são considerados **intratáveis**
- ▶ não é fácil dar bons exemplos de problemas não polinomiais: suspeita-se que o problema do circuito hamiltoniano, o problema da fatoração, o problema da mochila valiosa, e muitos outros não sejam polinomiais
- ▶ mas ainda estamos longe da comprovação dessas suspeitas

Problemas insolúveis

- ▶ além dos problemas não polinomiais, existem problemas **insolúveis**, também conhecidos como **indecidíveis**
- ▶ para esses problemas, não existe algoritmo algum, como por exemplo, o problema das equações diofantinas

Problemas polinomialmente verificáveis

- ▶ agora, devemos restringir nossa atenção aos problemas tais que é fácil reconhecer uma solução de uma instância do problema quando se está diante de uma
- ▶ um problema X é **polinomialmente verificável** se é possível verificar, em tempo polinomial, se uma suposta solução de uma instância de X é de fato uma solução
- ▶ ou seja, se existe um algoritmo que, ao receber uma instância I de X e uma suposta solução S de I , responde “sim” ou “não” conforme S seja ou não solução de I , e consome tempo limitado por um polinômio no tamanho de I para responder “sim”
- ▶ assim, as soluções são curtas, ou seja, o tamanho de uma solução é polinomial no tamanho da instância

Classe NP

- ▶ o conjunto dos problemas polinomialmente verificáveis constitui a **classe NP** de problemas
- ▶ “NP” não é abreviatura de “não polinomial”, mas sim de “*nondeterministic polynomial*”
- ▶ um problema X é **polinomialmente verificável** se é possível verificar, em tempo polinomial, se uma suposta solução de uma instância de X é de fato uma solução

Exemplos

Os problemas na primeira das duas listas de exemplos acima pertencem à classe NP:

Equação do segundo grau: é fácil verificar se um dado inteiro x satisfaz a equação $ax^2 + bx + c = 0$: basta calcular o número $a \cdot x \cdot x + b \cdot x + c$ e compará-lo com zero; se x satisfaz a equação então o número de dígitos de x não é maior que o número de dígitos de (a, b, c) e portanto os cálculos consomem tempo limitado por um polinômio no número de dígitos de (a, b, c) ; assim, o problema da equação do segundo grau pertence à classe NP

Exemplos

Os problemas na primeira das duas listas de exemplos acima pertencem à classe NP:

Fatoração: é fácil verificar se um dado número natural p é divisor de n : basta dividir n por p usando o algoritmo de divisão que todos aprendemos na escola; se p divide n então $p \leq n$ e portanto os cálculos consomem tempo polinomial no número de dígitos de n ; assim, o problema da fatoração está na classe NP

Exemplos

Os problemas na primeira das duas listas de exemplos acima pertencem à classe NP:

Circuito longo: o problema está na classe NP pois é possível verificar em tempo polinomial se uma dada sequência de vértices é um circuito do grafo e tem comprimento maior que k

Circuito hamiltoniano: o problema está em NP pois é possível verificar em tempo polinomial se uma dada permutação dos vértices é um circuito do grafo

Campo minado: o problema está em NP pois é possível verificar em tempo polinomial se uma dada distribuição das minas é consistente com a configuração dada

Exemplos negativos

- ▶ não é fácil dar bons exemplos de problemas que não estejam na classe NP
- ▶ um exemplo bobo é o problema de construir uma lista de todos os subconjuntos de $\{1, \dots, n\}$ uma vez dado n
- ▶ suspeita-se que muitos problemas de otimização não estão na classe NP
- ▶ no caso do problema do circuito máximo, por exemplo, embora seja fácil verificar se uma dada sequência de vértices é um circuito, não se conhece uma maneira eficiente de verificar que o circuito é máximo
- ▶ no entanto, o problema do máximo divisor comum está em NP pois está em P, como mostra o algoritmo de Euclides

P = NP ou P ≠ NP?

- ▶ é fácil entender que a classe NP inclui a classe P, ou seja, que todo problema polinomial é polinomialmente verificável
- ▶ parece que P é apenas uma pequena parte de NP
- ▶ surpreendentemente, ninguém encontrou ainda um problema de NP que não esteja em P, isto é, um problema polinomialmente verificável para o qual *não existe* algoritmo polinomial
- ▶ essa situação abre caminho para a suspeita de que talvez P seja igual a NP
- ▶ no entanto, a maioria dos especialistas não acredita nessa possibilidade
- ▶ o Instituto Clay de Matemática oferece um prêmio de um milhão de dólares pela solução da questão

Complexidade relativa de problemas

- ▶ o status de muitos problemas da classe NP é desconhecido: não se sabe se o problema está na classe P ou não
- ▶ então, é uma boa ideia investigar a complexidade *relativa* dos problemas da classe NP
- ▶ isto é, verificar se um dado problema Y é mais fácil ou mais difícil que um outro problema X (talvez mais bem-conhecido)

Exemplos

- ▶ informalmente, dizemos que um problema Y não é mais difícil que um problema X se Y for um “subproblema”, ou “caso particular”, de X
 - ▶ o problema do quadrado perfeito não é mais difícil que o problema da equação do segundo grau
 - ▶ o problema do circuito hamiltoniano não é mais difícil que o problema do circuito longo
 - ▶ o problema da soma de subconjunto não é mais difícil que o problema da mochila valiosa
 - ▶ o problema da cobertura pequena de um grafo não é mais difícil que o problema da clique grande

Redução polinomial

- ▶ para formalizar esse conceito, recorreremos à ideia de **redução polinomial**
- ▶ um problema Y **não é mais difícil que** – ou **é polinomialmente redutível a** – um problema X se qualquer algoritmo para X pode ser usado para resolver Y em três passos:
 1. transforme a instância dada de Y , em tempo polinomial, numa instância de X ,
 2. submeta a instância de X ao algoritmo para X , que produz uma solução S ,
 3. transforme S , em tempo polinomial, numa solução da instância de Y
- ▶ a expressão “tempo polinomial” é sempre relativa ao tamanho da instância de Y

Redução polinomial

- ▶ Se Y não é mais difícil que X e X está na classe P então Y também está na classe P
- ▶ ou seja, se Y não está na classe P e Y não é mais difícil que X então X também não está na classe P
- ▶ uma outra maneira de definir redução polinomial é: um problema Y é **polinomialmente redutível** a um problema X se existe um algoritmo que resolve Y usando uma subrotina hipotética que resolve X , de tal modo que se a subrotina for polinomial, o algoritmo para Y também é polinomial

NP, NP-difícil, NP-completo

- ▶ um problema X é **NP-difícil** se todos os problemas em NP não são mais difíceis que X
- ▶ ou seja, um problema é NP-difícil se for tão difícil quanto qualquer problema em NP
- ▶ um problema é **NP-completo** se for NP-difícil e estiver em NP
- ▶ a existência de problemas NP-completos é um fato surpreendente e fundamental
- ▶ tal fato foi demonstrado independentemente, por volta de 1970, por Steven Cook e Leonid Levin

NP, NP-difícil, NP-completo

- ▶ uma vez conhecido um problema NP-completo, o conceito de redução polinomial pode ser usado para mostrar que outro problema é NP-completo: se Y é um problema NP-completo e Y não é mais difícil que um problema X então X também é NP-completo
- ▶ por exemplo, são NP-completos os seguintes problemas:
 - ▶ soma de subconjunto
 - ▶ mochila valiosa
 - ▶ circuito longo
 - ▶ caminho longo
 - ▶ circuito hamiltoniano
 - ▶ clique grande
 - ▶ cobertura pequena
 - ▶ campo minado

NP, NP-difícil, NP-completo

- ▶ para provar que $P = NP$, basta encontrar um algoritmo polinomial para um único problema NP-completo
- ▶ mas ninguém conseguiu ainda realizar essa façanha ...

Exercícios

1. Descreva algoritmos polinomiais para cada um dos seguintes problemas: quadrado perfeito, equação do segundo grau, divisor comum grande, subsequência crescente longa e caminho curto
2. Descreva um algoritmo polinomial para o problema da cobertura pequena
3. [CLRS 16.2-2] O algoritmo 0-1-KNAPSACK é polinomial?
4. Descreva informalmente um algoritmo (não necessariamente polinomial) que resolva o problema do circuito hamiltoniano
5. Discuta a seguinte variante do problema do circuito hamiltoniano: Dado um grafo e uma lista de todas as permutações de vértices do grafo, encontrar um circuito hamiltoniano no grafo ou constatar que tal circuito não existe. Descreva um algoritmo polinomial que resolva o problema

Exercícios

6. Descreva informalmente um algoritmo (não necessariamente polinomial) que resolva o problema da fatoração
7. Suponha que as instâncias de um certo problema são pares de cadeias de caracteres. O tamanho de uma instância é então da forma $n + m$, onde n é o comprimento da primeira cadeia do par e m é o comprimento da segunda cadeia. Suponha que tenho um algoritmo para o problema que consome $\Omega(n 2^m)$ unidades de tempo. Mostre que o algoritmo não é polinomial
8. Suponha que cada instância de um certo problema consiste em um número natural k , um número natural n e um vetor $x_1, \dots, x_n$ cujos componentes são elementos do conjunto $\{1, 2, 3\}$. Um algoritmo que resolve qualquer instância do problema em $O(k n^2 \log n)$ unidades de tempo é polinomial?

Exercícios

9. Discuta a seguinte afirmação: “Se X é um problema não polinomial então todas as instâncias de X são difíceis (ou seja, não podem ser resolvidas em tempo polinomial)”
10. Mostre que os seguintes problemas estão na classe P: quadrado perfeito, equação do segundo grau, divisor comum grande, subsequência crescente longa e caminho curto
11. Mostre que os seguintes problemas estão na classe NP: divisor comum grande, soma de subconjunto, mochila valiosa, subsequência crescente longa, caminho curto, caminho longo, clique grande, cobertura pequena

Exercícios

12. Mostre que o problema do quadrado perfeito não é mais difícil que o problema da equação do segundo grau. Reciprocamente, mostre que o problema da equação do segundo grau não é mais difícil que o problema da raiz quadrada
13. Mostre que o problema do circuito hamiltoniano é polinomialmente redutível ao problema do circuito longo. Mostre a recíproca
14. Mostre que o problema da cobertura pequena de um grafo não é mais difícil que o problema da clique grande. (Comece por analisar a relação entre coberturas e conjuntos independentes. Um **conjunto independente** é um conjunto de vértices dois-a-dois não adjacentes.) Mostre a recíproca

Exercícios

15. No **problema do caixeiro viajante** um caixeiro viajante deve visitar n cidades. Modelando o problema como um grafo completo com n vértices, podemos dizer que o caixeiro viajante deseja fazer um percurso visitando cada cidade exatamente uma vez, terminando na cidade que ele iniciou. O custo de viajar de uma cidade i para uma cidade j é um número inteiro não negativo $c(i,j)$ e o caixeiro viajante deseja fazer um percurso cujo custo total é mínimo, onde o custo total é a soma dos custos individuais das arestas ao longo do percurso. O problema de decisão equivalente recebe, além do grafo completo G e da função custo c , um número inteiro k e pergunta se G tem um percurso do caixeiro viajante com custo no máximo k . Mostre que a versão de decisão do problema do caixeiro viajante é um problema NP-completo. *Sugestão*: mostre que o problema do caixeiro viajante não é mais difícil que o problema do circuito hamiltoniano.

Exercícios

16. Suponha que os algoritmos A e B transformam cadeias de caracteres em outras cadeias de caracteres. Suponha que A consome $O(n^2)$ unidades de tempo e B consome $O(n^4)$ unidades de tempo, sendo n é o número de caracteres da cadeia de entrada. Considere agora o algoritmo AB que consiste na composição de A e B, com B recebendo como entrada a saída de A. Qual o consumo de tempo de AB?
17. Mostre que todo problema da classe P é polinomialmente redutível ao problema do circuito hamiltoniano
18. Digamos que X é um problema NP-completo. É verdade que todas as instâncias de X são difíceis?

Exercícios

19. Suponha que $P \neq NP$. Quais das seguintes afirmações podemos inferir a respeito de um problema X ?
- (a) Se X é NP-completo então X não pode ser resolvido em tempo polinomial
 - (b) Se X está em NP então X não pode ser resolvido em tempo polinomial
 - (c) Se X está em NP mas não é NP-completo então X pode ser resolvido em tempo polinomial
 - (d) Se X está em P então X não é NP-completo